BCA-305: Design and Analysis of Algorithms

Course Weightage: 70 Marks (Block I: 40, Block II: 30)

Q1. What is Asymptotic Analysis of an algorithm?

Asymptotic analysis evaluates the performance of an algorithm in terms of input size (n) as it approaches infinity. It ignores hardware constraints and constant factors to focus on the growth rate of time or space complexity.

Key Concept / Formula: Growth Rate as $n \to \infty$

Q2. Explain Big-O, Omega (Ω), and Theta (Θ) notations.

Big-O provides an upper bound (worst-case), Omega (Ω) provides a lower bound (best-case), and Theta (Θ) provides a tight bound (when upper and lower bounds are the same) for the growth rate of an algorithm.

Key Concept / Formula: $O(g(n)) \leq f(n)$, $\Omega(g(n)) \geq f(n)$, $\Theta(g(n)) == f(n)$

Q3. Differentiate between Best, Average, and Worst-case behaviors.

Best-case is the minimum time required (ideal input). Worst-case is the maximum time required (most unfavorable input). Average-case is the expected time calculated over all possible inputs assuming a uniform probability distribution.

Key Concept / Formula: $T_best(n) \leq T_avg(n) \leq T_worst(n)$

Q4. Define Little-o and Little-omega (ω) notations.

Little-o provides a strict upper bound (the algorithm is strictly slower growing than the bound). Little-omega provides a strict lower bound. Unlike Big-O and Big-Omega, the bounds cannot be equal to the function's growth rate.

Key Concept / Formula: $f(n) = o(g(n))$ means $\text{limit}(f(n)/g(n)) = 0$ as $n \to \infty$

Q5. What are Time and Space Tradeoffs?

This principle states that you can often reduce a program's execution time by using more memory (space), or reduce memory usage by spending more time. For example, Dynamic Programming uses extra space (memoization) to drastically reduce time complexity.

Key Concept / Formula: Time vs. Space: O(n) space for O(1) time

Q6. How are Recurrence Relations used in algorithmic analysis?

Recurrence relations express the running time of a recursive algorithm in terms of the time taken by its smaller sub-problems. They are typically solved using the Substitution method, Recursion Tree method, or the Master Theorem.

Key Concept / Formula: $T(n) = aT(n/b) + f(n)$

Q7. Describe the Brute-Force algorithmic strategy.

Brute-force is a straightforward, exhaustive search approach that evaluates every possible solution to find the correct one. While simple to implement, it is usually highly inefficient for large datasets.

Key Concept / Formula: Example: Linear Search, Time Complexity O(n)

Q8. Explain the Divide and Conquer strategy.

This strategy recursively breaks down a problem into two or more sub-problems of the same type, solves them independently, and then combines their solutions to form the solution to the original problem.

Key Concept / Formula: Example: Merge Sort, $T(n) = 2T(n/2) + O(n)$

Q9. What is the Greedy Algorithm approach?

A greedy algorithm builds a solution piece by piece, always making the locally optimal choice at each step with the hope that these local optimums will lead to a global optimum. It does not reconsider previous choices.

Key Concept / Formula: Example: Dijkstra's Shortest Path

Q10. Discuss the Backtracking strategy.

Backtracking is a refined brute-force approach. It incrementally builds candidates to the solutions, and abandons a candidate ('backtracks') as soon as it determines that the candidate cannot possibly lead to a valid solution.

Key Concept / Formula: Example: N-Queens Problem

Q11. Explain the Branch and Bound technique.

Used primarily for optimization problems, Branch and Bound systematically explores the state space tree. It calculates a 'bound' (optimistic estimate) at each node and prunes branches that cannot yield a better solution than the best one found so far.

Key Concept / Formula: Pruning non-optimal branches

Q12. What are Pattern Matching and String Text Algorithms?

These algorithms find the occurrence(s) of a 'pattern' string within a larger 'text' string. Efficient algorithms like Knuth-Morris-Pratt (KMP) preprocess the pattern to avoid redundant comparisons.

Key Concept / Formula: KMP Algorithm: Uses Longest Prefix Suffix (LPS) Array

Q13. Define Heuristics and Numerical Approximation.

When exact solutions to complex problems (like NP-hard problems) take too long, heuristics are rules-of-thumb that find 'good enough' approximate solutions quickly. Numerical approximation estimates mathematical values using iterative computation.

Key Concept / Formula: Approximation Ratio $\rho \leq \varepsilon$

Q14. Compare Implementation Strategies for Graphs.

Graphs can be implemented using an Adjacency Matrix (a 2D array, fast for checking if an edge exists but uses $O(V^2)$ space) or an Adjacency List (an array of linked lists, slower for edge lookup but highly space-efficient for sparse graphs).

Key Concept / Formula: Matrix: A[i][j] vs List: Adj[i] -> j

Q15. Discuss Implementation Strategies for Trees.

Trees can be implemented dynamically using nodes containing data and pointers to children. Complete binary trees (like Heaps) can be efficiently implemented using arrays to save the overhead of storing pointers.

Key Concept / Formula: Array Representation: Left Child = 2i, Right = 2i+1

Q16. What are the common Performance Issues of Data Structures?

Performance bottlenecks arise from memory fragmentation, the overhead of dynamic memory allocation (pointers), and poor cache locality. Contiguous structures like arrays are more cache-friendly than scattered structures like linked lists.

Key Concept / Formula: Cache Locality & Pointer Overhead

Q17. Define the Complexity Class 'P'.

The Class P consists of all decision problems that can be solved by a deterministic Turing machine in polynomial time. These are generally considered 'tractable' or easily solvable problems.

Key Concept / Formula: Time Complexity = $O(n^k)$ for some constant k

Q18. Define the Complexity Class 'NP'.

The Class NP (Non-deterministic Polynomial time) consists of decision problems where, if a 'yes' answer is given, a deterministic Turing machine can verify the correctness of that answer in polynomial time.

Key Concept / Formula: $P \subseteq NP$ (Verification in Polynomial Time)

Q19. What is Polynomial Reducibility?

A problem A is polynomial-time reducible to problem B if an algorithm for B can be transformed to solve A with only a polynomial-time overhead. If we can quickly solve B, we can quickly solve A.

Key Concept / Formula: $A \leq_p B$

Q20. Explain NP-Completeness and NP-Hardness.

A problem is NP-Complete if it is in NP and every problem in NP is reducible to it. NP-Hard problems are at least as hard as the hardest problems in NP, but they do not themselves have to be in NP (they may not be verifiable in polynomial time).

Key Concept / Formula: Cook-Levin Theorem: SAT is NP-Complete